HW5

Theo McArn

March 2026

Question 1

1. **Why is Q-learning considered an off-policy control method?**

   Q-learning is considered to be off-policy because the policy is using an ϵ-greedy agent to collect samples however the Q-value function is updated based on the **maximum** Q-value of all of the possible next actions, which is a strictly greedy decision. Since the sampling policy, ϵ-greedy, is different from the policy being updated, i.e. the greedy target policy, Q-learning is considered off-policy.

2. **Suppose action selection is greedy. Is Q-learning then exactly the same algorithm as SARSA? Will they make exactly the same action selections and weight updates?**

   If action selection was greedy, then Q-learning and SARSA would both make the same weight updates and action selections. With Q-learning, the value of each state is updated based on the maximum Q-value of all of the next possible actions. With SARSA, the value of each state is updated based on the the Q-value of the next state-action pair. If action selection is greedy, than this is also the maximum Q-value of the next possible actions. As a result, if Q-values are updated in the same way, and both action selections are greedy, then it follows that all action selections will also be the same. Therefore, with this condition of greedy action selection, SARSA and Q-Learning will make exactly the same action selections and weight updates.

Question 2

Is there any situation (not necessarily related to this example) where the Monte-Carlo approach might be better than TD? Explain with an example, or explain why not.

TD learning is often preferable over Monte Carlo approaches because it converges faster, does online learning, and can handle episodes of long or infinite timesteps. These benefits stem from the fact that TD bootstraps, using its own value estimates to update rather than waiting for a complete episode return. However, this bootstrapping is only accurate when the agent and environment follow the Markov property (i.e., that each state and reward only depend on the previous state and action). In a lot of real-life examples, this assumption is not satisfied. Many cases are classified as partially observable MDPs or POMDPs. For example, if you were training an agent to play a card game against an opponent, the agent can only view its own cards and not the cards of the

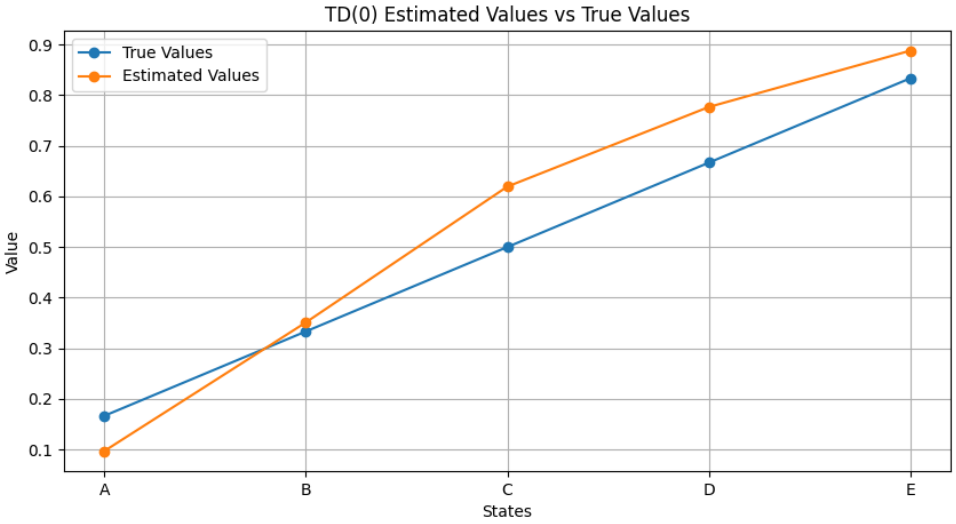

Figure 1: Value Estimation of Random Walk Policy with TD(0)

other player. This setup would violate the Markov property, and as a result, the TD method would provide extremely biased value estimates, since bootstrapping off inaccurate estimates causes errors to compound through successive updates. On the other hand, because Monte Carlo methods use the true sampled discounted return after each episode, their value estimates are unbiased.

Question 3

1. Value Estimation for the Random Walk Policy using TD(0) can be found in Figure 1.

2. (a) **Do you think the conclusions about which algorithm is better would be affected if a wider range of α values were used? Is there a different, fixed value of α at which either algorithm would have performed significantly better than shown? Why or why not?**

   Ultimately, the step size cannot improve estimation that significantly, and the differences between the TD(0) and MC estimations are mostly due to the amount of look ahead taken by the agent. In MC approach, the agent looks all the way to the end of the episode (in n-step TD terms, $n = \infty$), and in TD(0) the agent only looks at one step in front (n = 1). As you can see from Figure 7.2 in Sutton & Barto, the RMSE of the n-step TD with higher n values generally had a higher RMSE than the n-step TD agents with lower n values. Regardless of the choice for α. This plot shows that the performance of these agents is determined more by the number of steps looking ahead, rather than the value of α.

   (b) **In the right graph of the random walk task in Example 6.2, the RMS error of the TD method seems to go down and then up again, particularly at high**

α's. What could have caused this? Do you think this always occurs, or might it be a function of how the approximate value function was initialized?

This dip and then rise is a result of overshoot due to the large step size α. Even after an estimate close to the true one is reached, TD(0) continues to make large updates based on neighboring states and as a result, this causes increasing large oscillations around the true value estimate instead of steady convergence. These increased oscillations due to overshooting is what causes the error to increase. This overshoot is in part due to the accurate initialization as well as the large step size. Since all of the values were initialized to 0.5, which is very close to all of the true values, TD(0) quickly arrives at the true values but because of this sharp gradient it overshoots. If the initialied values were unrealistic, say 0 or 1, it would be a much steadier approach to the optimal values and this overshooting would be less intense.

Question 4

1. Comparison of SARSA, Expected-SARSA, Q-learning, and Monte-Carlo Agents on the Windy Gridworld Environment can be seen in in Figure 2.
2. Performance of Windy Gridworld with Knight Moves and No-Move options can be seen in Figure 3.
3. Performance of Windy Gridworkd with Random Wind can be seen in Figure 4

Question 5

1. A plot of the distribution of the estimated value of the start position of Windy Gridworld can be seen in Figure 5.
2. **Describe what you observe from your histograms. Comment on what they may show about the bias-variance trade-off between the different methods, and how it may depend on the amount of training that has already occurred.**

   This histogram clearly shows the bias-variance trade-off between these two methods. For TD(0), almost all samples of value estimation are the same with the exception of one or two outliers. However, this estimate, while it has very low variance is extremely biased and changes drastically when the number of evaluation episodes changes (shifts from -1.5 to -6 to -19). Since TD(0) updates its values using values from the previous iteration, there is inherent bias put into the estimate, which depends on the number of evaluation episodes.

   With Monte-Carlo on the other hand, there is a lot of variance, due to the nature of sampling from long rollouts of episodes. This distribution is heavily skewed in the negative direction. However, despite this variance, the estimate remains unbiased, regardless of the number of evaluation episodes run. The mean of the distribution remains around -20 for all three trials. This is because the Monte-Carlo method is not estimating new value based on old values, but instead estimates value based on true experience. Therefore the estimates are always unbias but due to the nature of random sampling, this introduces more variance.

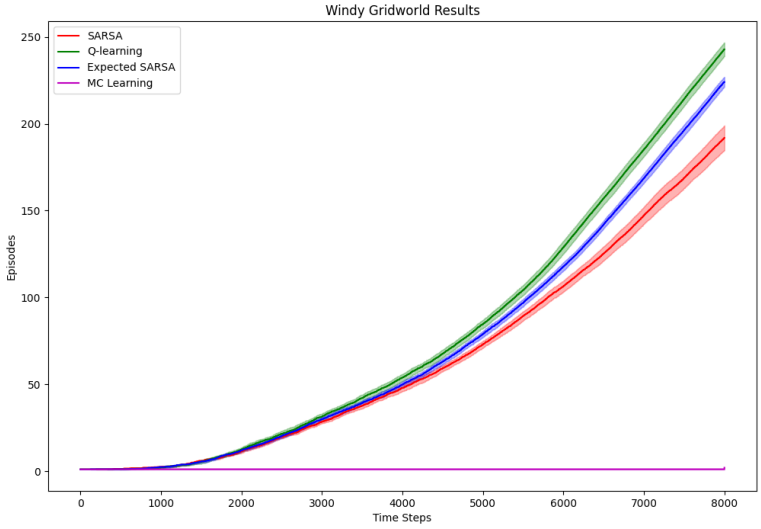

Figure 2: Comparison of Agents on Windy Gridworld

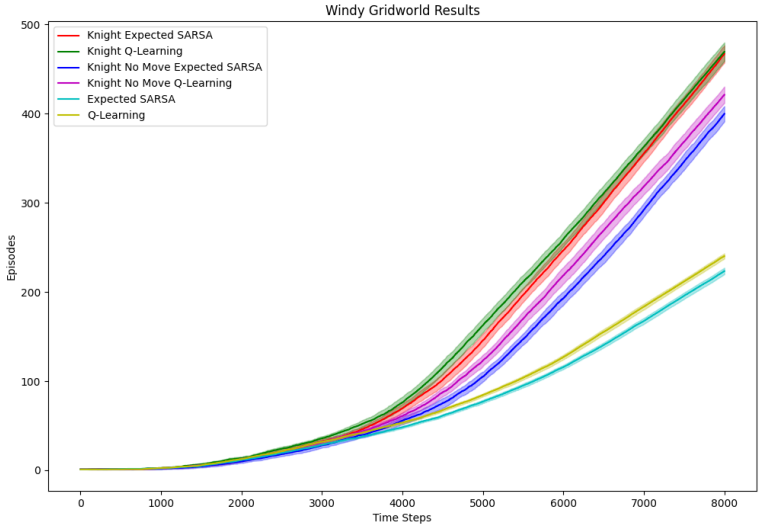

Figure 3: Windy Gridworld with Knights Moves and No Move Actions

3. **If we considered the scenario of control (i.e., we would use on-policy action-value methods, iteratively update the policy during training, and use it to generate the next training episode), would that change the results, and how? Please give a reasonable hypothesis.**

   Using on-policy action-value methods to iteratively update the policy, and then using this policy to rollout the next episode would result in better convergence. For TD(0) learning this would mean that the estimate while still biased, would be much closer to the true value function. And for MC, this would mean that the estimate is still unbiased but the variance is much lower due to convergence.

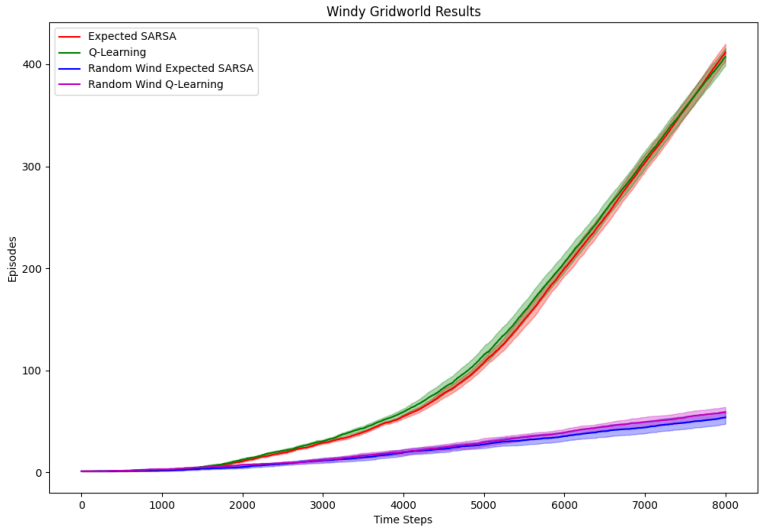

Figure 4: Windy Gridworld with Random Wind

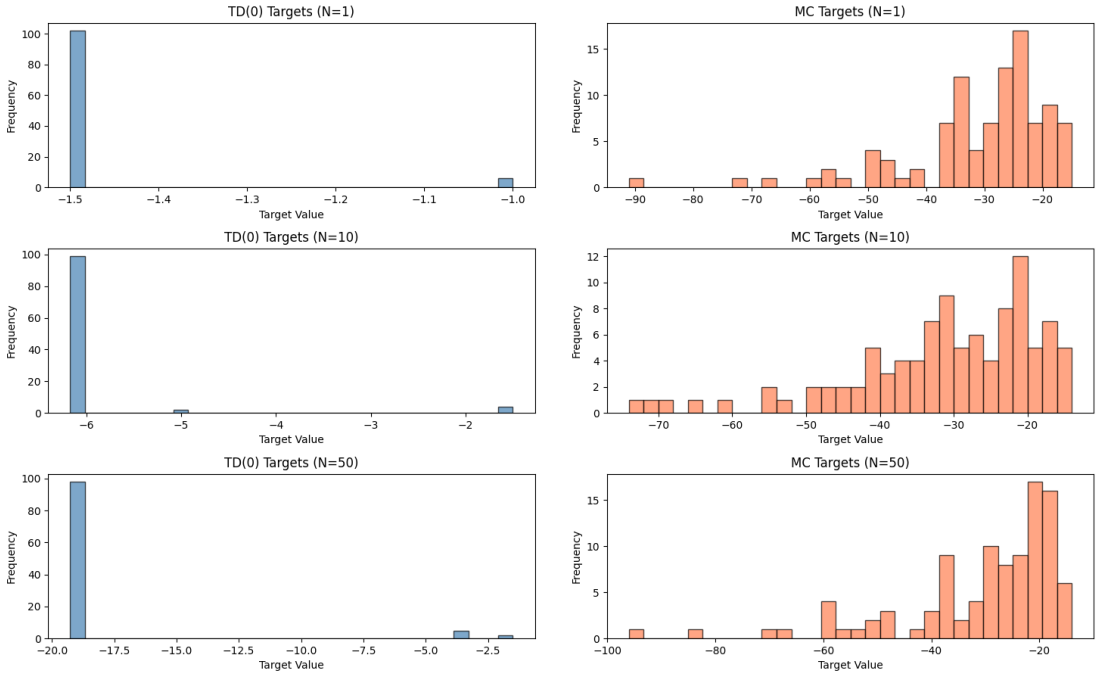

Figure 5: Estimated Value from the Start Position of Windy Gridworld